Student notes — Session 8 · Scope, call-by-value and recursion

Slides:
[`Class_08_Scope_call_by_value_recursion.pdf`](../slides/Class_08_Scope_call_by_value_recursion.pdf).
These notes are the notes pane of the student copy of the deck.

Slide 1 — Quiz 2: 25 minutes

This slide was on the screen during Quiz 2. Your marked paper comes back next week, on Blackboard.

Quiz 2 covered classes 3 to 6. Quiz 3 is on **Friday 16 October** and covers classes 7 to 10: functions, today's class, and arrays.

Slide 2 — Scope, call-by-value and recursion

The programs of this class are on Blackboard: Week 4 module, *Class 8 code*. It is a zip file. Unzip it and open the folder in Visual Studio Code. Every file has a comment at the top: what the program does, how to run it, and a list called **Try** with small changes to make. Make them.

How to run a program. Same as every week. In the terminal type `gcc -Wall name.c -o name` and press Enter, then `./name` (or `.\name` in Windows PowerShell) and press Enter.

Today answers three questions about functions: where does a variable live, what does a function get, and can a function call itself?

Slide 3 — What Tuesday gave us

What you need from Tuesday:

- A function has a **name**, **parameters** and a **return type**: `int square(int x)`.
- `return` hands **one** value back to the caller.
- The call is replaced by that value: `square(5)` becomes `25`.
- `void` means the function returns nothing.
- Write the function above `main`.

Slide 4 — Recap: what is on the screen?

Try to answer before you look at the next slide.

Slide 5 — Recap: what is on the screen?

The screen shows `25`. `square(3)` is replaced by 9 and `square(4)` by 16, so `total` is 9 + 16 = 25. The file is `01_recap_square.c`.

Slide 6 — Today's roadmap

Three parts:

1. **Scope**: where does a variable live?
2. **Call-by-value**: what does a function get?
3. **Recursion**: a function that calls itself.

Slide 7 — Scope

Part 1: where does a variable live?

Slide 8 — A variable lives inside its braces

A variable made inside a function lives **only** in that function. It is made when the function starts, and it is gone when the function ends. The part of the program where a name can be used is called its **scope**.

Slide 9 — main cannot see inside square

The file `02_bug_scope.c`. It does **not** compile. gcc says:

```
error: 'result' undeclared (first use in this function)
```

and points at the `printf` in `main`. The box `result` was made inside `square`, so it lives only there. `main` has no box called `result`.

The fix: keep the value that comes back. In `main`, write `int r = square(5);` and print `r`. Only the **value** comes back from a function, never the box.

Slide 10 — Two boxes with the same name

Try to answer before you look at the next slide. `main` has an `x`. `f` has an `x` too.

Slide 11 — Two boxes with the same name

The screen shows:

```
main: x = 5
in f: x = 100
main: x = 5
```

`main`'s `x` and `f`'s `x` are two **different** boxes. They have the same name, but each lives in its own function. What `f` does with its `x` cannot touch `main`'s `x`. The file is `03_same_name.c`.

Slide 12 — One name, two boxes

`main`'s `x` lives in `main`. `f`'s `x` lives in `f`, and it is made again every time `f` is called.

This is good news: inside a function you can choose names freely. They cannot clash with the names in other functions.

Slide 13 — The same rule for loops

The file `04_bug_block_scope.c`. It does **not** compile: `error: 'i' undeclared`. The `i` made in the brackets of the `for` lives only inside the loop. After the loop, it is gone.

You saw the same error in class 6, when `int sum = 0;` was written inside the loop. Same rule.

If you need `i` after the loop, make it before the loop: `int i;` and then `for (i = 1; i <= 3; i++)`.

Slide 14 — The scope rule

The rule in one sentence: **a name lives from the line that makes it to the `}` that closes its block.**

- Made in a function: gone when the function ends.
- Made in a loop: gone when the loop ends.

- Parameters too: the `x` in `square(int x)` lives only in `square`.

Slide 15 — Global variables: you will see them

A variable made **outside** every function, at the top of the file, is a **global** variable. Every function can see it and change it. In `05_global.c`, `tick` adds one to `count` three times, and `main` prints `count = 3`.

You will see globals in other people's code. **For now, do not use them in your own programs.** When every function can change a box, it is hard to find which one changed it. Send values in with parameters, and send them back with `return`.

Slide 16 — Call-by-value

Part 2: what does a function really get?

Slide 17 — A function gets a copy

When you call a function, the **value** of each argument is copied into the parameter. The parameter is a new box, inside the function. If the function changes it, it changes the copy. The original stays the same.

Think of giving a friend a photocopy of your notes. They can write all over the photocopy. Your notes stay clean.

This is called **call-by-value**: C always passes values, never the boxes themselves.

Slide 18 — Predict: does x change?

Try to answer before you look at the next slide. `x` is 5. `double_it` doubles its parameter `n`.

Slide 19 — Run it: x is still 5

The screen shows:

```
inside: n = 10
after: x = 5
```

`n` became 10, but `x` is still 5: `n` was a copy. The file is `06_double_it.c`.

The compiler does not warn you about this. No error, no warning, even with `gcc -Wall -Wextra`. You have to know it.

Slide 20 — Two boxes, side by side

The trace, one step at a time. Step 1, the call: the value of `x`, 5, is copied into `n`. Now there are two boxes, both holding 5.

Slide 21 — Two boxes, side by side

Step 2, `n = n * 2;`: `n` becomes 10. `x` is in another box, in `main`. Nothing touches it.

Slide 22 — Two boxes, side by side

Step 3, the function ends: `n` is gone. Back in `main`, `x` is 5. It was 5 the whole time.

step	**`main`: `x`**	**`double_it`: `n`**
the call	5	5

step	main: x	double_it: n
`n = n * 2;`	5	10
function ends	5	gone

Slide 23 — Even with the same name?

Try to answer before you look at the next slide. This time the parameter is called `x`, the same name as the variable in `main`.

Slide 24 — Even with the same name?

The screen shows `inside: x = 6`, then `after: x = 5`. The same name does not make it the same box. The parameter `x` lives in `add_one`, and it is a copy. The file is `07_add_one_same_name.c`.

Slide 25 — The fix: return the new value

To get a new value back, **return** it. In `08_double_return.c`, `double_it` returns `n * 2`, and `main` keeps it:

```
x = double_it(x);
```

The screen shows `after: x = 10`. The function does not change `x`. `main` changes `x`, with the value that came back.

If you write only `double_it(x);`, the returned value is thrown away and `x` stays 5.

Slide 26 — Predict: swap

Try to answer before you look at the next slide. `swap` swaps `a` and `b` with a third box, `temp`. Are `x` and `y` swapped after the call?

Slide 27 — swap does not swap

The screen shows:

```
in swap: 7 3
in main: 3 7
```

Inside `swap`, `a` and `b` were swapped. But they were copies. `x` and `y` in `main` were never touched. The file is `09_swap_fails.c`.

Slide 28 — Changing the caller's boxes needs pointers

`return` does not help here: it gives back **one** value, and a swap must change **two**. The real fix uses **pointers**, in class 18, after the midterm. Until then: a function gets copies, and `return` is the way back.

Slide 29 — Call-by-value in one slide

- **The call:** the value is copied into the parameter.
- **Inside:** changes change the copy.
- **The way back:** `return`, one value.

Slide 30 — Recursion

Part 3: a function that calls itself.

Slide 31 — A function may call itself

Some problems contain a smaller problem of the same kind. To count down from 3: say 3, then count down from 2. To count down from 2: say 2, then count down from 1. It must stop somewhere: at 0, say Go!

A function that calls itself is called **recursive**. Think of Russian dolls: inside each doll there is a smaller one, until the smallest, which does not open.

Slide 32 — countdown.c

The file `10_countdown.c`. `countdown(3)` prints:

```
3
2
1
Go!
```

Inside `countdown` there is a call to `countdown`. That is recursion.

Slide 33 — The base case: when to stop

The lit lines are the **base case**: when `n` is 0, print Go! and stop. No more calls. `return;` with nothing after it, in a `void` function, means "stop here and go back".

Slide 34 — The step closer

The lit line, `countdown(n - 1)`, is the **recursive call**: the same job, one smaller. Each call is closer to the base case: 3, 2, 1, 0.

Slide 35 — countdown(3), call by call

`countdown(3)` prints 3 and calls `countdown(2)`. `countdown(2)` prints 2 and calls `countdown(1)`. Each call has its **own** `n`, like any parameter. The call with 3 is not finished when the call with 2 starts: it is waiting.

Slide 36 — countdown(3), call by call

call	prints	then calls
`countdown(3)`	`3`	`countdown(2)`
`countdown(2)`	`2`	`countdown(1)`
`countdown(1)`	`1`	`countdown(0)`
`countdown(0)`	`Go!`	nothing: base case

Four calls of one function, each with its own `n`.

Slide 37 — Every recursive function has two parts

- **Base case:** a small case you answer directly. No more calls.
- **Step closer:** call yourself with a smaller problem.

Without a base case, or with a step that does not get closer, the function never stops calling itself. Slide 50 shows what happens then.

Slide 38 — Factorial two ways

Left: factorial with a loop, as in class 6, inside a function (`12_factorial_loop.c`). Right: factorial with recursion (`11_factorial_rec.c`). Both print `4! = 24` and `5! = 120`.

The recursive one reads like the maths: 4! = 4 × 3!, and 1! = 1.

Slide 39 — factorial: the base case

The base case: if `n` is 1 or less, return 1. The `<=` also covers 0, and 0! is 1 too.

Slide 40 — factorial: the step closer

The step closer: `return n * factorial(n - 1);`. The multiplication cannot happen yet: it must **wait** for the answer of the smaller call.

Slide 41 — factorial(4): down and back up

Going **down**: `factorial(4)` needs `4 * factorial(3)`. It cannot multiply yet, so it waits.

Slide 42 — factorial(4): down and back up

`factorial(3)` waits for `factorial(2)`, which waits for `factorial(1)`. `factorial(1)` is the base case: it returns 1 at once.

Slide 43 — factorial(4): down and back up

Coming back **up**: `factorial(2)` gets 1 and returns 2 × 1 = 2. `factorial(3)` gets 2 and returns 3 × 2 = 6.

Slide 44 — factorial(4): down and back up

`factorial(4)` gets 6 and returns 4 × 6 = 24. Four calls down, four returns up. All the multiplications happen on the way back up.

call	waits for, or returns
`factorial(4)`	`4 * factorial(3)`
`factorial(3)`	`3 * factorial(2)`
`factorial(2)`	`2 * factorial(1)`
`factorial(1)`	returns 1
`factorial(2)`	2 × 1: returns 2
`factorial(3)`	3 × 2: returns 6
`factorial(4)`	4 × 6: returns 24

Practise this table on paper. A question like this can come in Quiz 3.

Slide 45 — See it happen: the code

The file `13_factorial_trace.c`: the same factorial, with a `printf` when each call starts and another when it returns.

Slide 46 — See it happen: the output

The screen shows:

```
factorial(4) starts
factorial(3) starts
factorial(2) starts
factorial(1) starts
```

```
factorial(1) returns 1
factorial(2) returns 2
factorial(3) returns 6
factorial(4) returns 24
Answer: 24
```

It is the table from slide 44.

Slide 47 — Predict: print after the call

Try to answer before you look at the next slide. Like countdown, but the `printf` comes **after** the recursive call.

Slide 48 — Predict: print after the call

The screen shows `1`, `2`, `3`, one per line. On the way down, nothing is printed: each call first calls the smaller one. The numbers come out on the way back up, smallest first. The file is `14_count_up.c`. Move the `printf` above the call and it prints `3`, `2`, `1`.

Slide 49 — No base case: what happens?

Try to answer before you look at the next slide.

Slide 50 — No base case: it crashes

The file `15_bug_no_base_case.c`. gcc warns first:

```
warning: infinite recursion detected [-Winfinite-recursion]
```

It still builds a program. When you run it, the numbers fly past: 3, 2, 1, 0, -1, -2, ... Every call takes a little memory, and a waiting call does not give it back. After about 43,000 calls this memory runs out and Windows stops the program. **There is no message:** it just ends.

If you want proof, in PowerShell type `$LASTEXITCODE` and press Enter just after it stops. You see `-1073741571`: Windows' code for "stack overflow".

Read your warnings. gcc told you about this before you ran it.

Slide 51 — Your turn: sum_to(n)

Write `int sum_to(int n)` with recursion. `sum_to(4)` is 4 + 3 + 2 + 1 = 10. First answer two questions: what is the base case, and what is the step closer? Then look at the next slide.

Slide 52 — sum_to: one answer

```
int sum_to(int n) {
    if (n == 0) {
        return 0;
    }
    return n + sum_to(n - 1);
}
```

Base case: the sum of nothing, `sum_to(0)`, is 0. Step closer: `n` plus the sum of the smaller numbers. `sum_to(4)` is 10 and `sum_to(100)` is 5050. The file is `16_sum_to.c`.

If you wrote `if (n == 1) return 1;` as the base case, it works for 1 and up. But then `sum_to(0)` never reaches the base case: 0, -1, -2, ... and it crashes like slide 50.

Slide 53 — One more: power

`power(base, exp)` in `17_power_rec.c`. Base case: anything to the power 0 is 1. Step closer: `base * power(base, exp - 1)`. The screen shows `2 to the 10 = 1024` and `3 to the 4 = 81`.

Slide 54 — Why recursion can be slow

Fibonacci numbers: 0, 1, 1, 2, 3, 5, 8, 13, ... each is the sum of the two before it. The recursive version in `18_fib_calls.c` is short, but every call makes **two** calls, and they ask the same questions again and again. `fib(30)` is 832040, and it took 2,692,537 calls. A loop needs about 30 steps. Try `fib(40)` and watch how long you wait.

Slide 55 — Loop or recursion?

Everything in this class can also be written with a loop. Recursion is short when a problem holds a smaller copy of itself; you will meet such problems later in the course. For now, the goal is to **read** a recursive function and **trace** it on paper.

Slide 56 — Today in one slide

- A variable lives inside its `{ }`: that is its **scope**.
- The same name in two functions: two different boxes.
- A function gets a **copy** of each value.
- To send a new value back: `return` it.
- Recursion: a **base case** and a **step closer**.

Slide 57 — Before Tuesday

- **Knowledge check 8** on Blackboard covers this class. Three attempts, the best one is kept, and it does not count toward your grade.
- **Project 1** (Blackjack) is due **Sunday 11 October, 23:59**, on Blackboard. It asks for functions. Start early: a program that draws one card and prints it is a good first step.
- **Quiz 3 is on Friday 16 October**, the first 25 minutes of the class, on paper. It covers classes 7 to 10: functions, scope, call-by-value, recursion, and arrays.
- Today's code is on Blackboard, Week 4 module, *Class 8 code*. Make the "Try" changes in each file.

Next class: **arrays**. Many values under one name.